Лабораторная работа 5.

Разработка модульного приложения «Алгоритмическая обработка числовых рядов»

Цель:

Разработать **модульное** консольное приложение на Python, которое формирует числовые ряды, вычисляет базовые статистические показатели и реализует алгоритм скользящей средней.

Задачи:

1. Освоить **декомпозицию** задачи на независимые модули.
2. Научиться формировать массивы числовых данных (случайные, детерминированные, с шумом).
3. Реализовать **вручную** (без `statistics`, `numpy`, `pandas`) расчёт среднего, медианы, размаха.
4. Реализовать **эффективный** алгоритм скользящей средней через бегущую сумму.
5. Освоить обработку ошибок через `raise` / `try-except` / `from err`.
6. Научиться оформлять CLI через `argparse` и документировать код.

Задание:

Разработать модули:

- *Формирование массивов числовых данных для обработки*
- *Расчёт базовых статистических показателей (среднее, медиана, размах)*
- *Программная реализация алгоритма вычисления скользящей средней*

Структура проекта

```
num_series_app/
├── num_series/
│   ├── __init__.py # экспорт публичного API пакета
│   ├── generator.py # формирование числовых рядов
│   ├── stats.py # среднее, медиана, размах
│   └── moving_average.py # SMA, центрированное MA, EMA
│
├── tests/
│   ├── __init__.py
│   ├── test_generator.py
│   ├── test_stats.py
│   ├── test_moving_average.py
│   └── test_cli.py
├── conftest.py        ← НОВОЕ (настройка путей pytest)
├── pytest.ini         ← НОВОЕ (конфигурация pytest)
├── run_self_tests.py  ← НОВОЕ (запуск без pytest)
├── main.py  # CLI: связывает всё вместе
└── README.md # как запустить + примеры
```

⚠ **Важно:** папка `num_series/` и файл `main.py` лежат **на одном уровне**. Файл `__init__.py` внутри пакета обязателен.

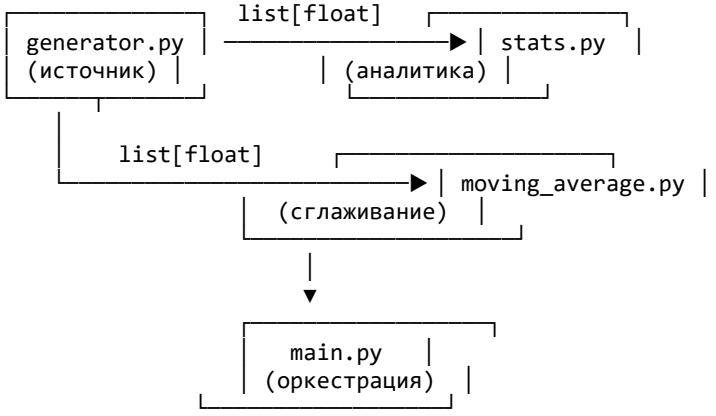
1. Создайте папку Лабораторная работа 5_ИвановИИ

2. Откройте VS Code для разработки проекта

2.1.Архитектура проекта

На примере разработанного проекта, создать свой проект по варианту. Далее описывается проект нулевого варианта.

При проектировании архитектуры предполагается следующее разделение ответственности
Прежде чем писать код — спроектируйте схему:



2.2.Определены следующие структуры данных

Что	Вход	Выход
Генераторы	n: int, ...	list[float]
Статистика	Sequence[float]	float / dict[str, float]
Скользящая средняя	Sequence[float], window: int	list[float]

2.3.Описание структуры проекта

Структура проекта должна содержать следующие модули

```
num_series_app/
├── num_series/
│   ├── __init__.py          # экспорт публичного API пакета
│   ├── generator.py  # формирование числовых рядов
│   ├── stats.py             # среднее, медиана, размах
│   └── moving_average.py # SMA, центрированное MA, EMA
```

Эта часть проекта будет добавлена после разработки основной части проекта

```
├── tests/
│   ├── __init__.py
│   ├── test_generator.py
│   ├── test_stats.py
│   ├── test_moving_average.py
│   └── test_cli.py
├── conftest.py      ←    (настройка путей pytest)
├── pytest.ini       ←     (конфигурация pytest)
├── run_self_tests.py  ←        (запуск без pytest)
├── main.py                  # CLI: связывает всё вместе
```

└── README.md # как запустить + примеры

⚠ **Важно:** папка num_series/ и файл main.py лежат **на одном уровне**. Файл __init__.py внутри пакета обязателен.

3. Разработка

3.1. Модуль генерации данных (generator.py)

Начнем разработку с генерированного источника данных — самая «нестабильная» часть программы. Сегодня это случайные числа, завтра — чтение из файла, послезавтра — из API. Изолировав его, мы защищаем остальную программу от изменений.

Начинаем с **validate**

Цель: прежде чем что-то генерировать, убедиться, что запрошено адекватное количество точек.

```
def _validate_n(n: int) -> None:
 """Проверка, что n — положительное целое число."""
 try:
  is_positive_int = (n > 0) and (int(n) == n)
 except (TypeError, ValueError) as err:
  raise ValueError(
   f"n должно быть положительным целым числом "
   f"(получено: {n!r}; причина: {err.args[0]})"
  ) from err

 if not is_positive_int:
  raise ValueError(
   f"n должно быть положительным целым числом (получено: {n!r})"
  )
```

Создадим три генератора случайных чисел

```
import math
import random
from typing import List, Optional


def random_series(n: int,
        low: float = -100.0,
        high: float = 100.0,
        seed: Optional[int] = None) -> List[float]:
 """Ряд из n равномерно распределённых случайных чисел на [low, high]."""
 _validate_n(n)
 if low >= high:
  raise ValueError(f"low должно быть меньше high (получено: {low} >= {high})")
 rng = random.Random(seed)
 return [rng.uniform(low, high) for _ in range(n)]


def sine_series(n: int,
       amplitude: float = 1.0,
```

```
        frequency: float = 1.0,
        noise: float = 0.0,
        seed: Optional[int] = None) -> List[float]:
    """Синусоидальный ряд с аддитивным шумом (для наглядной проверки MA)."""
    _validate_n(n)
    rng = random.Random(seed) // random.Random(seed) создаёт локальный генератор.
                                Глобальный random.seed(...) влияет на все модули
                                программы — это «загрязняет» чужое состояние.
                                Локальный — изолирован.
    return [
      amplitude * math.sin(2 * math.pi * frequency * i / n)
      + (rng.uniform(-noise, noise) if noise else 0.0)
      for i in range(n)
    ]
def trend_series(n: int,
        start: float = 0.0,
        step: float = 1.0,
        noise: float = 0.0,
        seed: Optional[int] = None) -> List[float]:
    """Линейный тренд (полезно, чтобы увидеть, что MA смещается)."""
    _validate_n(n)
    rng = random.Random(seed)
    return [
      start + step * i + (rng.uniform(-noise, noise) if noise else 0.0)
      for i in range(n)
    ]
```

3.2. Модуль статистики (`stats.py`)

Контракт и валидация

```
from typing import Dict, Sequence
def _ensure_not_empty(data: Sequence[float]) -> None:
    if not data:
        raise ValueError("Ряд пуст: расчёт статистики невозможен")
```

Все публичные функции вызывают валидатор **первой строкой** — это принцип **fail fast**.

Реализация :

```
def mean(data: Sequence[float]) -> float:
    """Среднее арифметическое."""
    _ensure_not_empty(data)
    return sum(data) / len(data)


def median(data: Sequence[float]) -> float:
    """Медиана (устойчива к выбросам)."""
    _ensure_not_empty(data)
    s = sorted(data)
    k = len(s) // 2
    if len(s) % 2:
        return float(s[k])
    return (s[k - 1] + s[k]) / 2.0
```

```
def spread(data: Sequence[float]) -> float:
  """Размах ряда: max - min."""
  _ensure_not_empty(data)
  return float(max(data) - min(data))


def summarize(data: Sequence[float]) -> Dict[str, float]:
  """Сводный отчёт: count, mean, median, spread, min, max."""
  _ensure_not_empty(data)
  return {
    "count": len(data),
    "mean": mean(data),
    "median": median(data),
    "spread": spread(data),
    "min": float(min(data)),
    "max": float(max(data)),
  }
```

3.3. Модуль скользящей средней (moving_average.py**)**

Теория:
Что такое **скользящая средняя (SMA)**
Дан ряд $x_0, x_1, \ldots, x_{n-1}$ и окно w. SMA в точке i — это среднее по w последним точкам:
SMA[i] = (x[i] + x[i+1] + … + x[i+w-1]) / w
Длина результата — $n - w + 1$.

Функция валидации check_window(window, n):

- Window — целое ≥ 1;
- window ≤ n (иначе окно больше ряда).

SMA (обязательное)—simple_moving_average(data, window).

```
  for i in range(len(data) - window + 1):
    result.append(sum(data[i:i+window]) / window)
```

✓**Нужно**— через бегущую сумму (O(n)):

```
running = sum(data[:window])
result = [running / window]
for i in range(window, len(data)):
  running += data[i] - data[i - window]
  result.append(running / window)
```

Длина результата: len(data) - window + 1.

Самостоятельное задание:

1. centered_moving_average — окно нечётное, выход выровнен по центру.
2. exponential_moving_average(data, alpha):
EMA[0] = data[0]
EMA[i] = alpha * data[i] + (1 - alpha) * EMA[i-1], 0 < alpha ≤ 1

✓ **Контрольная точка 3.**

```
from num_series.moving_average import simple_moving_average
simple_moving_average([1, 2, 3, 4, 5], 3)  # → [2.0, 3.0, 4.0]
simple_moving_average([1, 2], 3)      # → ValueError
```

3.4. Экспорт API — num_series/__init__.py

```
"""Пакет алгоритмической обработки числовых рядов.
Публичный API пакета:
 Генерация ряда:
  random_series, sine_series, trend_series

 Статистика:
  mean, median, spread, summarize
 Скользящие средние:
  simple_moving_average,
  centered_moving_average,
  exponential_moving_average

Пример:
 >>> from num_series import sine_series, summarize, simple_moving_average
 >>> data = sine_series(20, noise=0.5, seed=42)
 >>> stats = summarize(data)
 >>> sma = simple_moving_average(data, 3)
"""

from .generator import random_series, sine_series, trend_series
from .stats import mean, median, spread, summarize
from .moving_average import (
 simple_moving_average,
 centered_moving_average,
 exponential_moving_average,
)

__all__ = [
 # генерация
 "random_series", "sine_series", "trend_series",
 # статистика
 "mean", "median", "spread", "summarize",
 # скользящие средние
 "simple_moving_average",
 "centered_moving_average",
 "exponential_moving_average",
]

__version__ = "1.0.0"
```

3.5. Файл main.py — точка входа CLI

```
"""Точка входа: демонстрация модульного приложения обработки числовых рядов.

Запуск:
 python main.py --kind sine --n 20 --window 3 --alpha 0.3 --seed 7
```

```
Параметры:
 --kind тип ряда: random | sine | trend  (по умолчанию: sine)
 --n   количество точек в ряде    (по умолчанию: 20)
 --window размер окна для скользящей средней  (по умолчанию: 3)
 --alpha  коэффициент сглаживания для EMA  (по умолчанию: 0.3)
 --seed зерно ГПСЧ для воспроизводимости   (по умолчанию: 42)
"""
                from __future__ import annotations

                import argparse
                import sys
                from typing import List

                from num_series import (
                 random_series, sine_series, trend_series,
                 summarize,
                 simple_moving_average, centered_moving_average, exponential_moving_average,
                )
                # ---------------------------------------------------------------------
                # Фабрика ряда
                # ---------------------------------------------------------------------

                def build_series(kind: str, n: int, seed: int) -> List[float]:
                 """Возвращает числовой ряд заданного типа.

                 kind = "random" — равномерное распределение на [-50, 50];
                 kind = "sine"  — синусоида с амплитудой 10, частотой 2 и шумом 1.5;
                 kind = "trend" — линейный тренд с шагом 1.0 и шумом 0.5.
                 """
                 if kind == "random":
                  return random_series(n, low=-50, high=50, seed=seed)
                 if kind == "sine":
                  return sine_series(n, amplitude=10, frequency=2, noise=1.5, seed=seed)
                 if kind == "trend":
                  return trend_series(n, start=0, step=1.0, noise=0.5, seed=seed)
                 raise ValueError(f"Неизвестный тип ряда: {kind!r}")

                # ---------------------------------------------------------------------
                # Печать
                # ---------------------------------------------------------------------

                def print_series(title: str, values: List[float], fmt: str = "{:>10.3f}") -> None:
                 """Печатает ряд в одну строку с выравниванием."""
                 print(f"\n{title} (n={len(values)}):")
                 print(" " + " ".join(fmt.format(v) for v in values))

                def print_header(kind: str, n: int) -> None:
                 print("=" * 60)
                 print(f"Тип ряда: {kind}, n = {n}")
                 print("-" * 60)
```

```
def print_stats(stats: dict) -> None:
 """Печатает блок базовых статистических показателей."""
 print(f"Среднее арифметическое : {stats['mean']:>10.4f}")
 print(f"Медиана    : {stats['median']:>10.4f}")
 print(f"Размах (max - min)  : {stats['spread']:>10.4f}")
 print(f"min / max    : {stats['min']:>10.4f} / {stats['max']:.4f}")

def print_footer() -> None:
 print("=" * 60)

# ---------------------------------------------------------------------------
# Разбор аргументов
# ---------------------------------------------------------------------------

def build_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(
  description="Алгоритмическая обработка числовых рядов",
  formatter_class=argparse.ArgumentDefaultsHelpFormatter,
 )
 parser.add_argument(
  "--kind", choices=["random", "sine", "trend"], default="sine",
  help="тип генерируемого ряда",
 )
 parser.add_argument(
  "--n", type=int, default=20,
  help="количество точек в ряде",
 )
 parser.add_argument(
  "--window", type=int, default=3,
  help="размер окна для скользящей средней",
 )
 parser.add_argument(
  "--alpha", type=float, default=0.3,
  help="коэффициент сглаживания для EMA, 0 < alpha <= 1",
 )
 parser.add_argument(
  "--seed", type=int, default=42,
  help="зерно ГПСЧ для воспроизводимости",
 )
 return parser

# ---------------------------------------------------------------------------
# main
# ---------------------------------------------------------------------------

def main(argv: List[str] | None = None) -> int:
 """Точка входа. Возвращает код выхода (0 — успех, 1 — ошибка)."""
 args = build_parser().parse_args(argv)

 try:
  data = build_series(args.kind, args.n, args.seed)
  stats = summarize(data)
```

```
 sma = simple_moving_average(data, args.window)
 ema = exponential_moving_average(data, args.alpha)
 cma = (
  centered_moving_average(data, args.window)
  if args.window % 2 == 1 else None
 )
except ValueError as err:
 # Понятное сообщение вместо «сырого» трейсбека
 print(f"Ошибка: {err}", file=sys.stderr)
 return 1

print_header(args.kind, args.n)
print_stats(stats)

print_series("Исходный ряд", data)
print_series(f"SMA (window={args.window})", sma)
print_series(f"EMA (alpha={args.alpha})", ema)
if cma is not None:
 print_series(f"Центрированное MA (window={args.window})", cma)

print_footer()
return 0

if __name__ == "__main__":
 raise SystemExit(main())
```

4. Запуск проекта

python main.py --kind sine --n 20 --window 3 --alpha 0.3 --seed 7

5. Расширим main.py без потери модульности:

```
# Экспорт в CSV
  parser.add_argument("--export", metavar="FILE", help="экспорт в CSV")

  # В main():
  if args.export:
   import csv
   with open(args.export, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["i", "value"])
    for i, v in enumerate(data):
     w.writerow([i, v])
   print(f"Экспортировано в {args.export}")
```

Или вывод в JSON:

```
    import json
    if args.json:
     print(json.dumps({"data": data, "stats": stats, "sma": sma}, indent=2))
```

6. Если проект уже лежит в папке **Лабораторная работа 5_ИвановИИ** и вы хотите связать его с удаленным *репозиторий*

bash

cd <путь_к_вашему_проекту>

git init
git remote add origin <URL_вашего_репозитория>
git commit -m "Добавлена 5 лабораторная работа"
git branch -M main //если вы не находитесь в master
git push -u origin main

7. Расширение проекта: сохранение в JSON и чтение из файла

Расширим проект

```
num_series_app/
├── num_series/
│   ├── __init__.py          ← + экспорты новых функций
│   ├── generator.py
│   ├── stats.py
│   ├── moving_average.py
│   ├── io_json.py           ← НОВОЕ: сохранение и чтение JSON
│   └── io_file.py           ← НОВОЕ: чтение из txt/csv
├── main.py
└── README.md
```

Ключевой принцип модульности

Каждый модуль знает о других модулях только через их публичные функции и общий тип данных — `Sequence[float] → list[float]`**.**

Это означает, что:

- `stats.py` можно заменить на `numpy` — `main.py` не заметит;
- `generator.py` можно заменить на чтение из CSV — остальные модули не заметят;
- каждый модуль тестируется **изолированно**.

7.1. Файл num_series/io_json.py — сохранение и чтение JSON

"""Сохранение и чтение числовых рядов в формате JSON.

Формат файла:

```
{
  "meta": {
    "kind": "sine",
    "n": 20,
    "seed": 42,
    "created": "2025-01-15T10:30:00"
  },
  "data": [0.0, 8.09, 9.51, ...]
}
```

Метаданные опциональны — при чтении они не обязательны.
"""

```
from __future__ import annotations
import json
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional, Union

PathLike = Union[str, Path]
# ---------------------------------------------------------------------------
# Сохранение
# ---------------------------------------------------------------------------
```

```
def save_series_json(
    data: List[float],
    path: PathLike,
    meta: Optional[Dict[str, Any]] = None,
    indent: int = 2,
) -> None:
        """Сохраняет числовой ряд в JSON-файл.

        Args:
            data:   числовой ряд (непустой список float/int).
            path:   путь к файлу (создастся или перезапишется).
            meta:   произвольные метаданные (kind, seed, ...). Добавится
                    автоматически поле "created" с текущим временем.
            indent: отступ для читаемости (None — компактный JSON).

        Raises:
            ValueError: если data пуст или содержит нечисловые элементы.
        """
    if not data:
        raise ValueError("Ряд пуст: нечего сохранять")

    # Проверка типов — иначе json.dump упадёт с непонятной ошибкой
    for i, v in enumerate(data):
        if not isinstance(v, (int, float)) or isinstance(v, bool):
            raise ValueError(
                f"Элемент data[{i}] = {v!r} не является числом"
            )

    payload: Dict[str, Any] = {
        "meta": {
            "n": len(data),
            "created": datetime.now().isoformat(timespec="seconds"),
            **(meta or {}),
        },
        "data": list(data),
    }

    file_path = Path(path)
    try:
        file_path.parent.mkdir(parents=True, exist_ok=True)
        with file_path.open("w", encoding="utf-8") as f:
            json.dump(payload, f, ensure_ascii=False, indent=indent)
    except OSError as err:
        raise ValueError(f"Не удалось записать файл {file_path}: {err}") from err
# ---------------------------------------------------------------------------
# Чтение
# ---------------------------------------------------------------------------

def load_series_json(path: PathLike, with_meta: bool = False):
    """Читает числовой ряд из JSON-файла.
    Поддерживает два формата:
        1. {"data": [...], "meta": {...}}  — «полный» формат (наш)
        2. [...]                           — «голый» список чисел
    Args:
        path:      путь к файлу.
        with_meta: если True, возвращает кортеж (data, meta);
                   если False — только список чисел.
```

```
    Returns:
        list[float] или (list[float], dict).

    Raises:
        ValueError: если файл не найден, не JSON, нет поля data,
                    data не список или содержит нечисловые элементы.
    """
    file_path = Path(path)
    if not file_path.exists():
        raise ValueError(f"Файл не найден: {file_path}")

    try:
        with file_path.open("r", encoding="utf-8") as f:
            payload = json.load(f)
    except json.JSONDecodeError as err:
        raise ValueError(
            f"Файл {file_path} не является корректным JSON: "
            f"{err.msg} (строка {err.lineno})"
        ) from err
    except OSError as err:
        raise ValueError(f"Не удалось прочитать файл {file_path}: {err}") from err

    # Поддержка «голого» списка
    if isinstance(payload, list):
        data, meta = payload, {}
    elif isinstance(payload, dict):
        if "data" not in payload:
            raise ValueError(
                f"В файле {file_path} нет поля 'data'"
            )
        data = payload["data"]
        meta = payload.get("meta", {}) or {}
    else:
        raise ValueError(
            f"Некорректная структура JSON в {file_path}: "
            f"ожидался список или объект с 'data'"
        )

    if not isinstance(data, list):
        raise ValueError(
            f"Поле 'data' в {file_path} должно быть списком, "
            f"а не {type(data).__name__}"
        )
    if not data:
        raise ValueError(f"Ряд в файле {file_path} пуст")

    for i, v in enumerate(data):
        if not isinstance(v, (int, float)) or isinstance(v, bool):
            raise ValueError(
                f"Элемент data[{i}] = {v!r} в {file_path} не является числом"
            )

    result = [float(v) for v in data]
    return (result, meta) if with_meta else result
```

7.2. Файл num_series/io_file.py — чтение «текстовых» файлов

Числа могут лежать не только в JSON. Поддержим два «человеческих» формата: по одному числу на строку (txt) и через запятую/точку с запятой/пробел (csv).

```
"""Чтение числовых рядов из простых текстовых файлов.
Поддерживаемые форматы:
   .txt  — по одному числу на строку (пустые строки и #-комментарии пропускаются)
   .csv  — числа через запятую / точку с запятой / пробел / табуляцию
           (первая строка, начинающаяся с "value" или "x", считается заголовком)
"""
from __future__ import annotations
import re
from pathlib import Path
from typing import List, Union

PathLike = Union[str, Path]
_SEPARATORS = re.compile(r"[,;\s]+")

def load_series_txt(path: PathLike) -> List[float]:
   """Читает числа из .txt — по одному на строку."""
   file_path = Path(path)
   if not file_path.exists():
      raise ValueError(f"Файл не найден: {file_path}")

   result: List[float] = []
   try:
      with file_path.open("r", encoding="utf-8") as f:
         for lineno, raw in enumerate(f, start=1):
            line = raw.strip()
            if not line or line.startswith("#"):
               continue
            try:
               result.append(float(line))
            except ValueError as err:
               raise ValueError(
                  f"{file_path}:{lineno}: '{line}' не является числом"
               ) from err
   except OSError as err:
      raise ValueError(f"Не удалось прочитать файл {file_path}: {err}") from err

   if not result:
      raise ValueError(f"Файл {file_path} не содержит чисел")
   return result

def load_series_csv(path: PathLike) -> List[float]:
   """Читает числа из .csv — через разделитель, возможно, с заголовком."""
   file_path = Path(path)
   if not file_path.exists():
      raise ValueError(f"Файл не найден: {file_path}")

   result: List[float] = []
   try:
      with file_path.open("r", encoding="utf-8") as f:
         for lineno, raw in enumerate(f, start=1):
            line = raw.strip()
            if not line or line.startswith("#"):
```

```
                continue

            tokens = [t for t in _SEPARATORS.split(line) if t]
            if not tokens:
                continue
            # Пропускаем заголовок: первая строка, где первое поле не число
            try:
                values = [float(t) for t in tokens]
            except ValueError:
                if lineno == 1:
                    continue   # считаем это заголовком
                raise ValueError(
                    f"{file_path}:{lineno}: строка '{line}' содержит "
                    f"нечисловые значения"
                )
            result.extend(values)
    except OSError as err:
        raise ValueError(f"Не удалось прочитать файл {file_path}: {err}") from err

    if not result:
        raise ValueError(f"Файл {file_path} не содержит чисел")
    return result

def load_series_auto(path: PathLike) -> List[float]:
    """Автоматически выбирает парсер по расширению.
    .json        → load_series_json
    .txt, .dat   → load_series_txt
    .csv, .tsv   → load_series_csv
    иначе        → пробует csv, при ошибке — txt
    """
    from .io_json import load_series_json

    suffix = Path(path).suffix.lower()
    if suffix == ".json":
        return load_series_json(path)
    if suffix in {".txt", ".dat"}:
        return load_series_txt(path)
    if suffix in {".csv", ".tsv"}:
        return load_series_csv(path)
    try:
        return load_series_csv(path)
    except ValueError:
        return load_series_txt(path)
```

Обновлённый `num_series/__init__.py`

```
"""Пакет алгоритмической обработки числовых рядов."""
from .generator import random_series, sine_series, trend_series
from .stats import mean, median, spread, summarize
from .moving_average import (
    simple_moving_average,
    centered_moving_average,
```

```
    exponential_moving_average,
)
from .io_json import save_series_json, load_series_json
from .io_file import load_series_txt, load_series_csv, load_series_auto

__all__ = [
    # генерация
    "random_series", "sine_series", "trend_series",
    # статистика
    "mean", "median", "spread", "summarize",
    # скользящие средние
    "simple_moving_average", "centered_moving_average", "exponential_moving_average",
    # ввод-вывод
    "save_series_json", "load_series_json",
    "load_series_txt", "load_series_csv", "load_series_auto",
]
__version__ = "1.1.0"
```

---

4. Обновлённый `main.py`

Добавляем **два новых источника данных** и **экспорт**. Логика:

- если указан `--load PATH` — берём ряд из файла, `--kind/--n/--seed` игнорируются;
- иначе — генерируем как раньше;
- если указан `--save PATH` — сохраняем ряд в JSON после генерации.

```
"""Точка входа: обработка числовых рядов (генерация / чтение / сохранение).

Примеры:
    # Генерация + анализ
    python main.py --kind sine --n 20 --window 3 --seed 7

    # Генерация + сохранение в JSON
    python main.py --kind random --n 50 --save data/random.json

    # Загрузка из файла + анализ
    python main.py --load data/random.json --window 5
    python main.py --load data/series.csv  --window 5

    # Пайплайн: генерация → сохранение → чтение → анализ
    python main.py --kind sine --n 30 --save data/s.json
    python main.py --load data/s.json --window 3

    # Экспорт результата в CSV
    python main.py --kind trend --n 30 --export out.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import sys
from typing import List, Optional, Tuple
```

```
from num_series import (
    random_series, sine_series, trend_series,
    summarize,
    simple_moving_average, centered_moving_average, exponential_moving_average,
    save_series_json, load_series_auto,
)


# ---------------------------------------------------------------------
# Источники данных
# ---------------------------------------------------------------------
def build_series(kind: str, n: int, seed: int) -> List[float]:
    """Генерирует ряд заданного типа."""
    if kind == "random":
        return random_series(n, low=-50, high=50, seed=seed)
    if kind == "sine":
        return sine_series(n, amplitude=10, frequency=2, noise=1.5, seed=seed)
    if kind == "trend":
        return trend_series(n, start=0, step=1.0, noise=0.5, seed=seed)
    raise ValueError(f"Неизвестный тип ряда: {kind!r}")


def obtain_series(args: argparse.Namespace) -> Tuple[List[float], str]:
    """Возвращает (ряд, описание источника)."""
    if args.load:
        data = load_series_auto(args.load)
        return data, f"файл: {args.load}"
    data = build_series(args.kind, args.n, args.seed)
    return data, f"сгенерирован ({args.kind}, n={args.n}, seed={args.seed})"


# ---------------------------------------------------------------------
# Экспорт в CSV
# ---------------------------------------------------------------------
def export_series_csv(path: str, data: List[float]) -> None:
    """Экспорт ряда в CSV: столбцы i, value. Ошибки нормализуются в ValueError."""
    try:
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(["i", "value"])
            for i, v in enumerate(data):
                w.writerow([i, v])
    except OSError as err:
        raise ValueError(f"Не удалось записать CSV {path}: {err}") from err


# ---------------------------------------------------------------------
# Печать
# ---------------------------------------------------------------------
def print_series(title: str, values: List[float], fmt: str = "{:>10.3f}") -> None:
    print(f"\n{title} (n={len(values)}):")
    print("  " + " ".join(fmt.format(v) for v in values))
```

```
def print_header(source: str) -> None:
    print("=" * 60)
    print(f"Источник данных: {source}")
    print("-" * 60)


def print_stats(stats: dict) -> None:
    print(f"Среднее арифметическое : {stats['mean']:>10.4f}")
    print(f"Медиана                : {stats['median']:>10.4f}")
    print(f"Размах (max - min)     : {stats['spread']:>10.4f}")
    print(f"min / max              : {stats['min']:>10.4f} / {stats['max']:.4f}")


def print_footer() -> None:
    print("=" * 60)


# ---------------------------------------------------------------------
# CLI
# ---------------------------------------------------------------------
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Алгоритмическая обработка числовых рядов",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
        epilog="Пример: python main.py --kind sine --n 20 --save data/s.json",
    )
    # источники
    parser.add_argument("--kind", choices=["random", "sine", "trend"], default="sine",
                        help="тип генерируемого ряда")
    parser.add_argument("--n", type=int, default=20,
                        help="количество точек в ряде")
    parser.add_argument("--seed", type=int, default=42,
                        help="зерно ГПСЧ для воспроизводимости")
    parser.add_argument("--load", metavar="PATH",
                        help="загрузить ряд из файла (.json/.txt/.csv); "
                             "отменяет --kind/--n/--seed")
    # экспорт
    parser.add_argument("--save", metavar="PATH",
                        help="сохранить ряд в JSON-файл")
    parser.add_argument("--export", metavar="FILE",
                        help="экспортировать ряд в CSV (i, value)")
    # анализ
    parser.add_argument("--window", type=int, default=3,
                        help="размер окна для скользящей средней")
    parser.add_argument("--alpha", type=float, default=0.3,
                        help="коэффициент сглаживания для EMA, 0 < alpha <= 1")
    return parser


def main(argv: Optional[List[str]] = None) -> int:
```

```
args = build_parser().parse_args(argv)

# --- 1. Получаем ряд ---
try:
    data, source = obtain_series(args)
except ValueError as err:
    print(f"Ошибка получения ряда: {err}", file=sys.stderr)
    return 1

# --- 2. Сохраняем (до анализа — данные на диске даже при ошибке анализа) ---
if args.save:
    meta = {"source": source}
    if not args.load:
        meta.update({"kind": args.kind, "seed": args.seed})
    try:
        save_series_json(data, args.save, meta=meta)
        print(f"[сохранено] {args.save} ({len(data)} точек)")
    except ValueError as err:
        print(f"Ошибка сохранения: {err}", file=sys.stderr)
        return 1

if args.export:
    try:
        export_series_csv(args.export, data)
        print(f"[экспортировано] {args.export} ({len(data)} точек)")
    except ValueError as err:
        print(f"Ошибка экспорта: {err}", file=sys.stderr)
        return 1

# --- 3. Анализ ---
try:
    stats = summarize(data)
    sma = simple_moving_average(data, args.window)
    ema = exponential_moving_average(data, args.alpha)
    cma = (
        centered_moving_average(data, args.window)
        if args.window % 2 == 1 else None
    )
except ValueError as err:
    print(f"Ошибка анализа: {err}", file=sys.stderr)
    return 1

# --- 4. Вывод ---
print_header(source)
print_stats(stats)

print_series("Исходный ряд", data)
print_series(f"SMA (window={args.window})", sma)
print_series(f"EMA (alpha={args.alpha})", ema)
if cma is not None:
    print_series(f"Центрированное MA (window={args.window})", cma)
```

```
    print_footer()
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```